

ARCHITECTURE PLAN

PDF ingestion and pgvector

How the four policy PDFs become searchable vectors in PostgreSQL, using free, local Ollama models, with replace-on-change behavior and an audited, repeatable CLI.

PLAN ONLY / NOT YET IMPLEMENTED

PROJECT

AI-assisted SupportOps

PLAN DATE

October 2, 2026

SCOPE

PDF ingestion, embeddings, retrieval check

What has been decided

The goal is a free, self-contained demo. Documents are extracted, chunked, embedded locally, and stored next to the existing structured data, with every chunk traceable to a source page.

TOPIC	DECISION
Embedding provider	Ollama, local and free. Default model <code>nomic-embed-text</code> (768 dimensions).
Changed PDF	Replace. A new checksum at the same <code>source</code> replaces its pages, chunks and embeddings. No version history.
Search method	Exact vector search with cosine distance. No ANN index at demo size.
Keyword search	Deferred. Add only if retrieval misses exact terms.
Execution	Explicit CLI (<code>npm run ingest:pdf</code>), following the CSV pattern. Not triggered by API startup or an HTTP route.
Chat model	Chat remains mocked in this milestone. A real chat model is added later and has no database impact.

Out of scope: chat integration, hybrid search, an ANN index, OCR for scanned PDFs, document version history, and an HTTP trigger.

Idempotent stages with clear boundaries

```
CLI -> PDF import service -> extractor -> chunker -> embedding client (Ollama)
      \-> PostgreSQL (documents, pages, chunks, embeddings)
```

- **Extractor and chunker** are pure and database-free, like the CSV batch parser, so they are easy to unit test.
- **Embedding client** is a thin HTTP wrapper around Ollama and is mocked in service tests.
- **Import service** owns the transaction and audit lifecycle.
- **CLI** handles options, reading files, and reporting.

The existing schema already models the content: `documents` (identity, checksum, type, date), `document_pages` (page text) and `document_chunks` (derived text tied to a page). Embeddings are stored separately so text remains the source of truth.

Build order

1. **Infrastructure.** Switch the database image to a pgvector-enabled Postgres 17 image and add an Ollama service. The existing data volume is preserved.
2. **Migration 005.** Run `CREATE EXTENSION vector`. Add `document_chunk_embeddings` with `chunk_id` (cascade delete), `model`, `dimensions` and `embedding vector(768)`, unique on (`chunk_id`, `model`). Add a PDF run-audit table mirroring `data_import_runs`. Migrations 001 to 004 are not modified.
3. **Extract.** Read the four PDFs under `datasets/rag_documents/`, compute SHA-256, take title and date from the visible text, and write one `document_pages` row per page.
4. **Chunk.** Split by heading or paragraph at about 200 to 500 tokens with a small overlap. Store the heading in `metadata`. Each chunk keeps its `page_id` for citations.
5. **Embed.** Call Ollama per chunk, in batches. Record model and dimension. Fail the run if Ollama is unreachable or a vector has the wrong size.
6. **Load.** In one transaction per document: skip if the checksum is unchanged; otherwise delete the old pages, chunks and embeddings, then insert the new ones. Commit the audit status with the data; record failures separately, as the CSV import does.
7. **Retrieval check.** Add a query function returning top-k chunks with document and page, using cosine distance (`<=>`). It is not exposed through the chat API in this milestone.
8. **Documentation.** Update the data model, schema, ERD, server README and the report PDF.

Content handling

- The four PDFs are short, single-page, text-extractable policies, so no OCR is needed. The design should still tolerate multi-page documents.
- Chunk quality matters more than model choice at this scale. Keep headings with their paragraphs.
- Replace-on-change reuses the existing cascade deletes (document to pages to chunks) and extends them to embeddings.
- An unchanged checksum is a no-op, which makes repeated runs safe.

Storage, algorithm, and metric

- **Image.** The official `postgres:17` image does not include `pgvector`. Use `pgvector/pgvector:pg17`, then enable the extension through a migration.
- **Separate embeddings table.** It lets you re-embed with a new model without touching the text, and the model name is stored on every row.
- **Dimension.** It is fixed per column and set by the model. A model change means re-embedding.
- **Distance metric.** Match what the model was trained for, usually cosine (`<=>`, `vector_cosine_ops`). The query operator must match an index operator class.

APPROACH	WHEN TO USE	TRADE-OFFS
Exact scan	Demo size (dozens of chunks). Chosen.	Perfect recall, no index to tune, slows as data grows.
HNSW	Thousands of chunks or more.	Best recall and speed, can build on an empty table. Slower builds and more memory. Tune <code>m</code> , <code>ef_construction</code> , <code>hnsw.ef_search</code> .
IVFFlat	Large data with fast build needs.	Faster build, lower recall, needs existing data to choose lists.

`pgvector` indexes on the `vector` type support up to 2000 dimensions. Larger vectors need shortening or `halfvec`. Filtering by `document_type` alongside ANN search can reduce recall unless tuned. Hybrid search (full-text plus vector, for example with reciprocal rank fusion) is a later option for exact terms.

Why Ollama

FACTOR	HOSTED API	LOCAL (OLLAMA)
Cost	Per-call pricing, API key.	Free.
Privacy	Text leaves your environment.	Stays local.
Quality	Strong, especially large models.	Good for short policy text. Varies by model.
Operations	No infrastructure.	Extra service, model download, CPU or GPU load.
Stability	Models can be deprecated.	Model is pinned and reproducible.

Embedding model names, dimensions and pricing change over time. Verify against current provider documentation before committing.

Two different jobs

	EMBEDDING MODEL	CHAT (GENERATIVE) MODEL
Examples	nomic-embed-text, mxbai-embed-large	llama3.2, qwen2.5, mistral
Output	A fixed-length vector. No readable text.	Generated text, token by token.
Purpose	Find relevant chunks by meaning.	Write the answer from the question and retrieved chunks.
Size and speed	Hundreds of MB, fast, fine on CPU.	Gigabytes, slower, benefits from a GPU.
Runs	At ingestion and for each query.	Only when answering.
Database impact	Fixes the vector dimension. Changing it means re-embedding.	None. Swap freely.

Question -> embedding model -> vector -> pgvector search -> top chunks
 Question + chunks -> chat model -> answer (with page citations)

This milestone needs only the embedding model. The chat model is added when the mocked chat is replaced, and it needs considerably more RAM.

08 / DOCKER COMPOSE SKETCH

Database and Ollama together

Illustrative only; the Compose file has not been changed.

```
services:
  db:
    image: pgvector/pgvector:pg17 # was postgres:17
    environment:
      POSTGRES_DB: ${DATABASE_NAME}
      POSTGRES_USER: ${DATABASE_USER}
      POSTGRES_PASSWORD: ${DATABASE_PASSWORD}
    ports:
      - "${DATABASE_PORT}:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  ollama:
    image: ollama/ollama
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    healthcheck:
      test: ["CMD", "ollama", "list"]
      interval: 10s
      retries: 10

  ollama-pull:
    # one-shot model download
    image: ollama/ollama
    depends_on:
      ollama:
        condition: service_healthy
```

```
environment:
  OLLAMA_HOST: ollama:11434
  entrypoint: ["sh", "-c", "ollama pull ${EMBEDDING_MODEL} && ollama pull ${CHAT_MODEL}"]
  restart: "no"

volumes:
  postgres_data:
  ollama_data:
```

- One Ollama service serves both models and loads each on demand.
- Models persist in `ollama_data`. The first start downloads a few GB.
- A 3B chat model such as `llama3.2` is about 2 GB and runs acceptably on CPU. NVIDIA GPU use needs the container toolkit and a Compose device reservation.
- A host-run Node server uses `http://localhost:11434`. A containerized server would use `http://ollama:11434`.
- For this milestone only the embedding model is required. The chat model can be added later.

How we will know it works

Verification

- Unit tests for the chunker, including overlap and page mapping.
- Service tests with a mocked embedding client: replace-on-change leaves no orphaned chunks or embeddings; an unchanged checksum is skipped; a mid-batch failure rolls back while the failed audit record is preserved.
- Live run against Ollama: the four PDFs ingest, a second run changes nothing, and expected questions (for example how long refunds take) retrieve the right document and page.
- Strict typecheck, build and full test suite under Node 22, since the installed system Node is 18.16.0.

Risks and notes

- The pgvector image change is one-time, and everyone cloning the repository must pull the Ollama model. Document this in the README.
- CPU embedding is slow but acceptable for four short PDFs.
- If the embedding model changes later, re-embed. The stored `model` column makes that a data job, not a redesign.
- Queries and documents must use the same embedding model and dimension.

Open items

- Confirm the exact embedding model (default `nomic-embed-text`).
- Decide later whether hybrid keyword search is needed after reviewing retrieval results.